

ML course season 2

Лекция: Подбор гиперпараметров и интерпретируемость моделей

Паша Кашмель

Курс по машинному обучению

22 апреля 2026 г.

План лекции

- 1 Гиперпараметры и кросс-валидация
- 2 Стратегии поиска гиперпараметров
- 3 Early Stopping и пайплайны
- 4 Интерпретируемость моделей
- 5 Permutation Importance и PDP
- 6 SHAP

Что такое гиперпараметры?

Параметры vs гиперпараметры

Параметры	Гиперпараметры
Учатся из данных	Задаются вручную
w, b в LR	C, γ в SVM
μ, σ^2 в NB	<code>max_depth</code> в дереве

Примеры гиперпараметров:

- Скорость обучения η
- Параметр регуляризации λ
- Количество соседей k (KNN)
- Глубина дерева, число деревьев

Почему «просто перебрать» — опасно?

Если подбираем гиперпараметры на **тестовой** выборке — она перестаёт быть независимой оценкой.

Получаем **оптимизм**: модель выглядит лучше, чем есть на самом деле.

Утечка данных (data leakage) в процессе выбора.

Кросс-валидация: KFold

Идея KFold ($k = 5$)

Делим данные на k равных частей (фолдов).

На каждой итерации:

- $k-1$ фолдов — обучение
- 1 фолд — валидация

Итоговая метрика: **среднее по k оценкам**.

Фолд 1 Фолд 2 Фолд 3 Фолд 4 Фолд 5

Итер. 1	обучение	валидация	обучение	обучение	обучение
Итер. 2	обучение	обучение	валидация	обучение	обучение
Итер. 3	обучение	обучение	обучение	валидация	обучение
Итер. 4	обучение	обучение	обучение	обучение	валидация
Итер. 5	обучение	обучение	обучение	обучение	обучение

StratifiedKFold

Для классификации: сохраняем **пропорции классов** в каждом фолде.

Важно при **дисбалансе классов**!

Nested CV: вложенная кросс-валидация

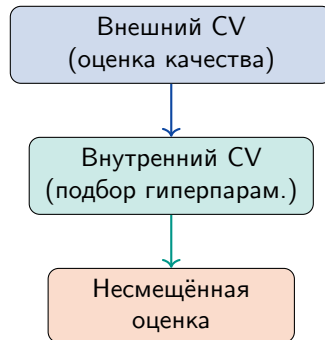
Проблема обычного CV:

Если мы выбираем гиперпараметры по CV на всех данных — CV-оценка **оптимистична** (мы уже «видели» тест).

Решение — два уровня CV:

- 1 **Внешний цикл:** оценка качества модели
- 2 **Внутренний цикл:** подбор гиперпараметров

Каждый внешний фолд: подбираем лучшие гиперпараметры на внутренних фолдах, затем оцениваем на внешнем.



На практике

Дорого вычислительно, но даёт **честную** оценку.

Grid Search и Random Search

Grid Search (перебор сетки)

Перебираем **все** комбинации из заданного набора значений.

- + Гарантированно найдём лучшее из сетки
- + Воспроизводимо
- Дорого: $|H_1| \times |H_2| \times \dots$ конфигураций
- Проклятие размерности

Random Search (случайный поиск)

Случайно сэмплируем конфигурации из заданных распределений.

- + Нашли хорошее решение быстрее
- + Можно задать непрерывные диапазоны
- + Легко параллелизовать
- Не гарантирует глобальный оптимум

Bergstra & Bengio (2012): при фиксированном бюджете Random Search превосходит Grid Search, если важны лишь некоторые гиперпараметры.

Bayesian Optimization: концепт

Идея: учимся предсказывать, *какие* гиперпараметры дадут хороший результат, по уже проведённым экспериментам.

Три шага:

- 1 Строим **суррогатную модель** (surrogate) функции качества: обычно Gaussian Process
- 2 **Acquisition function** выбирает следующую точку: баланс exploration / exploitation
- 3 Обновляем суррогат по новому результату

Инструменты: optuna, hyperopt, scikit-optimize



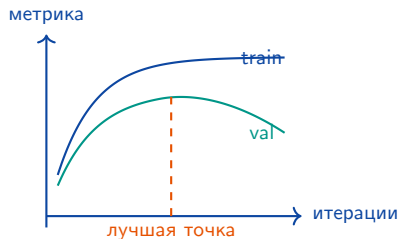
Early Stopping: когда хватит?

Проблема

Чем дольше тюним — тем больше рискуем **переобучиться к валидации**: случайно нашли конфигурацию, которая хороша именно на этом разбиении.

Критерии остановки:

- Бюджет времени / итераций исчерпан
- Улучшение метрики $< \varepsilon$ за N шагов
- Patience: нет улучшения за последние k итераций
- Доверительный интервал перестал меняться



Overfitting к валидации

Много итераций тюнинга = **утечка** через валидацию

Настройка пайплайна: препроцессинг + модель

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC()),
])

param_grid = {
    'svm__C': [0.1, 1, 10, 100],
    'svm__gamma': ['scale', 'auto', 0.01],
    'svm__kernel': ['rbf', 'linear'],
}

search = GridSearchCV(pipe, param_grid,
    cv=5, scoring='f1_macro', n_jobs=-1)
```

Зачем пайплайн?

- Гарантирует: нормировка обучается **только на train-фолде**
- Защита от **утечки данных** при CV
- Совместный подбор параметров препроцессинга и модели

Опасная ошибка

Применить StandardScaler ко **всем** данным до CV — это data leakage!

Трекинг экспериментов и выбор лучшей модели

```
import pandas as pd

results = pd.DataFrame(
    search.cv_results_
)[['params',
    'mean_test_score',
    'std_test_score',
    'rank_test_score']]

# Лучшие параметры
print(search.best_params_)
print(f"CV score: {search.best_score_:.4f}")

# Оценка на тесте - только один раз!
from sklearn.metrics import
↪ classification_report
y_pred = search.predict(X_test)
print(classification_report(y_test, y_pred))
```

Принципы трекинга

- Сохраняем **все** результаты, не только лучший
- Фиксируем `random_state`
- Смотрим на **std** — устойчивость важна
- Тест — **только финальная** оценка

Правило одного теста

Тестовая выборка — **не для итераций**. Смотрим ровно один раз после выбора модели.

Зачем нужна интерпретируемость?

Три главные причины

- ❶ **Доверие:** почему модель приняла решение? Пользователи и заказчики хотят объяснений.
- ❷ **Отладка:** нашли «странный» признак? Может быть утечка данных или артефакт!
- ❸ **Требования:** медицина, кредит, GDPR — право на объяснение решения.

Accuracy недостаточно

Модель с высокой точностью может:

- Использовать **утечку** данных
- Опирается на **артефакт** выборки
- Дискриминировать по **защищённым** признакам
- Быть нестабильной на **новых** данных

«A model that performs well but for the wrong reasons is dangerous.»

Global (глобальные)

Что важно для **модели в целом**?

Вопрос: Какие признаки в среднем влияют на предсказание?

Методы:

- Feature Importance (деревья)
- Permutation Importance
- PDP (Partial Dependence Plot)

Local (локальные)

Почему модель предсказала **именно это** для **конкретного** объекта?

Вопрос: Что повлияло на предсказание для примера x_i ?

Методы:

- SHAP values
- LIME
- Force plots

SHAP объединяет оба подхода: локальные объяснения с глобальным агрегированием.

Permutation Importance (PI)

Идея:

- 1 Обучаем модель, считаем базовую метрику S_0
- 2 Для каждого признака j : **перемешиваем** его значения случайно
- 3 Считаем новую метрику S_j
- 4 Важность: $PI_j = S_0 - S_j$

Преимущества

Работает с **любой** моделью. Независим от архитектуры. Прост в реализации.

Ограничения

Коррелированные признаки: если $x_1 \sim x_2$, перемешивание x_1 не «сломает» модель — она будет использовать x_2 .

PI занижает важность коррелированных признаков!

Решение: **conditional** PI или агрегирование групп.

PDP (Partial Dependence Plot)

Показывает **средний** эффект признака x_j на предсказание при маргинализации по всем остальным признакам:

$$\hat{f}_j(x_j) = \mathbb{E}_{\mathbf{x}_{\setminus j}} [\hat{f}(x_j, \mathbf{x}_{\setminus j})]$$

ICE (Individual Conditional Expectation)

То же самое, но **для каждого** объекта — отдельная кривая. Выявляет **взаимодействия** признаков, скрытые в среднем.

Ограничения PDP

- Среднее скрывает **гетерогенность**: разные группы объектов могут иметь разный эффект
- Не работает при **сильных взаимодействиях**: смотрим на ICE
- Предполагает **независимость** признаков при маргинализации

SHAP: идея вкладов признаков

SHAP (SHapley Additive exPlanations):

Вклад каждого признака в отклонение предсказания от среднего.

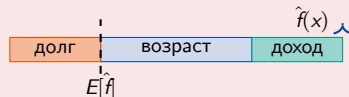
$$\hat{f}(x) = \underbrace{\mathbb{E}[\hat{f}]}_{\text{базовое}} + \sum_{j=1}^d \phi_j$$

Значение Шэпли ϕ_j — справедливое распределение «заслуги» из теории кооперативных игр.

Свойства:

- Локальная точность: сумма даёт предсказание
- Отсутствие фиктивных: нулевой вклад у неважных

Force plot (локальный)



Summary plot (глобальный)

Все объекты на одном графике: ось $X = \text{SHAP}$, цвет = значение признака

SHAP: чтение графиков

```
import shap

# Для деревьев (быстро)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Summary plot: глобальная важность
shap.summary_plot(shap_values, X_test)

# Force plot: один объект
shap.force_plot(
    explainer.expected_value,
    shap_values[0],
    X_test.iloc[0]
)
```

Как читать Summary Plot

- **Ось Y:** признаки (по убыванию важности)
- **Ось X:** SHAP-значение (+ = вверх по предсказанию)
- **Цвет:** величина признака (красный = высокое, синий = низкое)
- **Ширина:** плотность объектов

SHAP vs PI

SHAP: **локален** и работает с корреляциями корректнее. PI: проще и быстрее.

Часть 1: Гиперпараметры

- 1 **CV** — честная оценка без утечки
- 2 **Grid / Random** — стратегии поиска
- 3 **Bayesian Opt** — умный поиск
- 4 **Pipeline** — защита от leakage
- 5 **Early stopping** — против перетренинга

Часть 2: Интерпретируемость

- 1 **PI** — перемешиваем и смотрим
- 2 **PDP/ICE** — эффект признака
- 3 **SHAP** — вклады из теории игр
- 4 **Global vs Local** — разные вопросы

Домашнее задание

Выполнить практический ноутбук: подбор гиперпараметров + интерпретация на изученных моделях

Вопросы?

Спасибо за внимание!

Полезные ресурсы

`optuna.org` • `shap.readthedocs.io`
`scikit-learn.org/stable/modules/grid_search.html`
`christophm.github.io/interpretable-ml-book`